

Backend :

Express:

1. RestApi CRUD:

GET, POST, PATCH, DELETE:

Example for CRUD:

```
const express= require('express')    // importing the package express
```

```
const app= express()    // creating the server instant of the express
app.use(express.json())
```

```
// notes= title,description
const notes=[]
```

```
app.get("/notes", (req, res)=>{
  res.status(201).json({
    message: "retieved data successfully",
    notes: notes
  })
})
```

```
app.post("/notes", (req, res)=>{
  // console.log(req.body)
  notes.push(req.body)

  res.status(201).json({
    message: "Notes created correctly"
  })
})
```

```
app.post("/notes", (req, res)=>{
  // console.log(req.body)
  notes.push(req.body)

  res.status(201).json({
    message: "Notes created correctly"
  })
})
```

```
// /notes/:index
app.delete("/notes/:index", (req, res)=>{
  const index=req.params.index

  delete notes [ index ]

  res.status(201).json({
    message: "Notes deleted at index correctly"
  })
})
```

```
// /notes/:index
app.patch("/notes/:index", (req, res)=>{
  const index=req.params.index
  const description=req.body.description
  notes[index].description=description
  res.status(200).json({
    message: "Notes updated at index correctly"
  })
})
```

```
module.exports=app
```

2. Connection To MongoDB:

Code snippet to connect to the mongo:

```
const mongoose=require("mongoose")

async function connectDB(){
  await
mongoose.connect("mongodb+srv://youtube:XoCxJDw4WCdf0e8P@backendcomplete.mbv8lda
.mongodb.net/hello-db")
  console.log("Connected to the DB")
}
module.exports=connectDB;
```

3. Models/Schema in the MongoDB:

```
const mongoose=require('mongoose')

const noteSchema=new mongoose.Schema({
  title: String,
  description: String
})

const noteModel=mongoose.model("note",noteSchema)

module.exports=noteModel;
```

4. Usage of the MongoDB Connection in app.js

Code Snippet for the POST:

```
app.post("/notes",async (req,res)=>{
  const data=req.body
  await noteModel.create({
    title: data.title,
    description: data.description
  })
  res.status(201).send({
    message: "Notes created"
  })
})
```

GET all: const notes=await noteModel.find()

GET id : const id=req.params.id;
const notes = await noteModel.findById({_id:id})

DELETE id: const id = req.params.id;
await noteModel.findOneAndDelete({_id:id})

PATCH id : const id = req.params.id;
const des=req.body.description;
await noteModel.findOneAndUpdate({_id:id},{ description:des})

5. ImageKit (upload the image in the cloud and access)

```
src/services/storage.service.js
const {ImageKit}= require('@imagekit/nodejs')

const imagekit=new ImageKit({
  privateKey:process.env.IMAGEKIT_PRIVATE_KEY
})
```

```
async function uploadFile(buffer) {

  const result=await imagekit.files.upload({
    file:buffer.toString("base64"),
```

```

        fileName:"image.jpg"
    })

    return result;
}

module.exports=uploadFile;

app.js

const upload=multer({storage:multer.memoryStorage()})

app.post("/create-post",upload.single("image"),async (req,res)=>{
    console.log(req.body);
    console.log(req.file);

    const result= await uploadFile(req.file.buffer)

    console.log(result)

    const post=await postModel.create({
        image: result.url,
        caption: req.body.caption
    })
    return res.status(201).json({
        message:"succesfully uploaded",
        post:post
    })
})

```

Multer receives the uploaded image and stores it temporarily in memory as a buffer → ImageKit uploads the buffer to cloud storage → ImageKit returns the image URL → the image URL and caption are saved in MongoDB through the post model → later, the image can be accessed using the stored ImageKit URL.

6. Routes:
src/app.js:

```

const express=require('express');
const authRoutes=require('../src/routes/auth.route')
const postRoutes=require('../src/routes/post.route')
const cookieParser=require('cookie-parser')

const app=express();
app.use(express.json())
app.use(cookieParser())

app.use("/api/auth",authRoutes)
app.use("/api/posts",postRoutes)

module.exports=app;

src/routes/auth.route.js:
const express=require('express')
const router=express.Router();
router.post('/register',authController.registerUser)
module.exports=router;

```

```
src/routes/post.route.js:
const express=require('express')
const router=express.Router();
router.post('/register',postController.createPost)
module.exports=router;
```

```
src/controllers/auth.controller.js
async function registerUser(req,res){}
module.exports={registerUser}
```

```
src/controllers/post.controlle.js
async function createPost(req,res){}
module.exports={createPost}
```

Express Router: A built-in Express feature used to create separate route modules for different parts of an application. It helps organize and manage routes cleanly instead of putting all routes in one file.

Both consists of only the function it can be normal or async function

7. Authentication (JWT)
it is done sig the jwt secret

```
creating
token = jwt.sign({id:user._id},process.env.JWT_SECRET)
```

```
using
const decoded=jwt.verify(token,process.env.JWT_SECRET)
```

```
const user=await userModel.findOne({_id:decoded.id})
```

```
store in the cookie:
res.cookie("token", token);
```

cookie by using this code automatically the current token can be fetched:

```
const token=req.cookies.token;
```

flow:

Login → create JWT → store JWT in cookie → request → fetch token from cookie → verify JWT → get user.

Creating code snippet:

```
async function registerUser(req,res){
  const {username,email,password}=req.body;

  const isUserAlreadyExists=await userModel.findOne({
    email
  })

  if(isUserAlreadyExists)
  {
    return res.status(409).json({
      message:"User already exists"
    })
  }

  const user=await userModel.create({
    username,email,password
  })
}
```

```

const token=jwt.sign({
  id:user._id
},process.env.JWT_SECRET)

res.cookie("token",token)

res.status(201).json({
  message: "User registered successfully",
  user
})
}

```

Using code snippet:

```

router.post('/create', async (req, res)=>{

  const token=req.cookies.token;

  if(!token){
    return res.status(401).json({
      message:"Unauthorized"
    })
  }
  try{
    const decoded=jwt.verify(token,process.env.JWT_SECRET)

    const user=await userModel.findOne({
      _id:decoded.id
    })
    console.log(decoded)
    console.log(user)
  }catch(err)
  {
    return res.status(401).json({
      message:"Invalid token",
      err
    })
  }

  console.log(req.body)
  console.log(req.cookies)
  res.send("Dummy api")
})

```

8. Bcrypt (password hashing)

code snippet:

```

const bcrypt = require("bcrypt");
const hashedPassword = await bcrypt.hash(password, 10); // hash before storing
const isMatch = await bcrypt.compare(password, hashedPassword); // verify
password during usage(login)

```

Flow:

Password → bcrypt.hash() → hashed password → store in MongoDB → login → bcrypt.compare() → verify password.

9. jest and supertest to test the api

```

src/test/__app.test.js
const request=require('supertest')
const app=require("../app")

```

```

describe("GET /",()=>{

```

```

    it("should return 200 OK", async()=> {
      const res=await request(app).get("/");

      expect(res.statusCode).toBe(200);
      expect(res.body).toEqual({message:"opened the url"});
    })
  })
})

```

10. Validation (apply condition on the input data and validate it correctly)

```

const { body, validationResult } = require("express-validator");

```

```

function validateResult(req, res, next) {
  const errors = validationResult(req);

  if (!errors.isEmpty()) {
    return res.status(400).json({
      errors: errors.array()
    });
  }

  next();
}

```

```

const registerUserValidationRules = [
  body("username")
    .isString()
    .withMessage("Username must be a string")
    .isLength({ min: 3, max: 20 })
    .withMessage("Username must be between 3 and 20 characters"),

  body("email")
    .isEmail()
    .withMessage("Invalid email address"),

  body("password")
    .isLength({ min: 6 })
    .withMessage("Password must be at least 6 characters"),

  validateResult
];

```

```

module.exports = { registerUserValidationRules };

```

```

in app.js
app.post("/register", ValidationRules.registerUserValidationRules, (req, res)=>{
  const {username,email,password}=req.body;

  res.status(201).json({message:"User registered successfully",user:{
    username,email
  }})
})

```

Here it is an middleware :

```

ValidationRules.registerUserValidationRules

```

Library used so far:

- * Express: A Node.js framework used to build web servers and APIs.
- * Express Router: Used to organize routes into separate modules.
- * JSON Web Token (jsonwebtoken): Used to create and verify tokens for user authentication.
- * Mongoose: A library used to interact with MongoDB using schemas and models.

- * Cookie-parser: Middleware used to read and manage cookies in Express.
- * CORS: Allows or restricts requests between different domains.
- * Multer: Middleware used to handle file uploads in Express.
- * Nodemon: Automatically restarts the Node.js server when code changes.
- * Bcrypt is used to securely hash passwords before storing them in the database.

HTTP Status Code:

Here are the main HTTP status codes you should know for a backend interview:

1xx - Informational

- * 100: Continue
- * 101: Switching Protocols

2xx - Success

- * 200: OK - Request successful
- * 201: Created - Resource created successfully
- * 202: Accepted - Request accepted for processing
- * 204: No Content - Successful request with no response body

3xx - Redirection

- * 301: Moved Permanently
- * 302: Found - Temporary redirect
- * 304: Not Modified - Cached version can be used

4xx - Client Errors

- * 400: Bad Request - Invalid request/input
- * 401: Unauthorized - Authentication required/invalid
- * 403: Forbidden - Access denied
- * 404: Not Found - Resource doesn't exist
- * 405: Method Not Allowed - HTTP method not supported
- * 409: Conflict - Resource already exists/conflict
- * 422: Unprocessable Content - Validation failed
- * 429: Too Many Requests - Rate limit exceeded

5xx - Server Errors

- * 500: Internal Server Error - Server-side error
- * 501: Not Implemented - Server doesn't support the functionality
- * 502: Bad Gateway - Invalid response from another server
- * 503: Service Unavailable - Server temporarily unavailable
- * 504: Gateway Timeout - Another server took too long to respond

****For interviews, remember these most:**** `200, 201, 204, 400, 401, 403, 404, 409, 422, 500, 503`.